

Experiment No. – 10							
Date of Performance:	3rd Aug 2026						
Date of Submission:	4th Aug 2026						
Conceptual Understanding (02)	Execution (1.5)	Problem Solving & Debugging(02)	Professional Conduct & Lab Ethics(02)	Output/ Result Accuracy (1.5)	Documentation & Result Analysis (01)	Experiment Total (10)	Sign with Date

Experiment No. 10 Study of Multithreading

10.1 Aim: Implementation of multi-threading : Create a Java program that creates three threads: Thread 1 prints numbers from 1 to 5, Thread 2 prints alphabets from A to E and Thread 3 prints symbols (*) five times. Use the Thread class.

10.2 Course Outcome: Develop robust Java applications using exception handling and multithreading techniques

10.3 Learning Objectives: To understand and apply how the resources can be used more efficiently with help of Multithreading and Synchronization

10.4 Requirement: Java development kit (JDK as per OS)

10.5 Related Theory:

Multithreading in Java is a process of executing two or more threads simultaneously to maximum utilization of CPU. A thread is a lightweight sub-process, the smallest unit of processing. Multiprocessing and multithreading, both are used to achieve multitasking.

Java supports multithreading through the Thread class. Java Thread allows us to create a lightweight process that executes some tasks. We can create multiple threads in our program and start them.

However, we use multithreading rather than multiprocessing because threads use a shared memory area. They don't allocate a separate memory area so saves memory, and context-switching between the threads takes less time than process.

Multiprocessing in Java

Multiprocessing in Java is purely based on the number of processors available on the host computer. Every process initiated by the user is sent to the CPU (processor). It loads the registers on the CPU with the data related to the assigned process.

To perform multiprocessing in Java, the user needs one processor. Therefore, when the user requests the simultaneous execution of the second process, the alternate CPU core gets triggered and executes the process.

Multithreading in Java

Multithreading in Java is a similar approach to multiprocessing. However, there are some fundamental differences between the two. Instead of a physical processor, multithreading involves virtual and independent threads.

It assigns each process with one or more threads based on their complexity. Each thread is virtual and independent of the other. This makes process execution much safer. If a thread or two are terminated during an unexpected situation, the process execution will not halt.

Lifecycle of a Thread in Java

The lifecycle of each thread in Java has five different stages. You will look into each one of those stages in detail. The Stages of the Lifecycle are mentioned below.

- New
- Runnable
- Running
- Waiting
- Dead

New

The first stage is "New". This stage is where it initiates the thread. After that, every thread remains in the new state until the thread gets assigned to a new task.

Runnable

The next stage is the runnable stage. Here, a thread gets assigned to the task and sets itself for running the task.

Running

The third stage is the execution stage. Here, the thread gets triggered as control enters the thread, and the thread performs a task and continues the execution until it finishes the job.

Waiting

At times, there is a possibility that one process as a whole might depend on another. During such an encounter, the thread might halt for an intermediate result because of its dependency on a different process. This stage is called the Waiting Stage.

Dead

The final stage of the process execution with Multithreading in Java is thread termination. After it terminates the process, the JVM automatically declares the thread dead and terminates the thread.

This stage is known as the dead thread stage.

Thread class extends Object class and implements Runnable interface.

Threads can be created by using two mechanisms:

1. Extending the Thread class
2. Implementing the Runnable Interface

1. Thread creation by extending the Thread class

We create a class that extends the java.lang.Thread class. This class overrides the run() method available in the Thread class. A thread begins its life inside run() method. We create an object of our new class and call start() method to start the execution of a thread. Start() invokes the run() method on the Thread object.

```
class thread_pr extends Thread
{
    public void run()
    {
        try
        {
            System.out.println( "Thread " + Thread.currentThread().getId() + " is
running");
        }
        catch (Exception e)
        {
            System.out.println("Exception is caught");
        }
    }
}

public class Multithread
{
    public static void main(String[] args)
    {
        int n = 8;
        for (int i = 0; i < n; i++)
        {
            thread_pr object = new thread_pr();
            object.start();
        }
    }
}
```

Output

```
Thread 15 is running
Thread 14 is running
Thread 16 is running
Thread 12 is running
```

Thread 11 is running
Thread 13 is running
Thread 18 is running
Thread 17 is running

2. Thread creation by implementing the Runnable Interface

We create a new class which implements java.lang.Runnable interface and override run() method. Then we instantiate a Thread object and call start() method on this object.

class Run_pr implements Runnable

```
{
    public void run()
    {
        try
        {
            System.out.println("Thread " + Thread.currentThread().getId() + " is running");
        }
        catch (Exception e)
        {
            System.out.println("Exception is caught");
        }
    }
}

class Multithread
{
    public static void main(String[] args)
    {
        int n = 8;
        for (int i = 0; i < n; i++)
        {
            Thread object = new Thread(new Run_pr());
            object.start();
        }
    }
}
```

Output:

Thread 13 is running
Thread 11 is running
Thread 12 is running
Thread 15 is running
Thread 14 is running
Thread 18 is running
Thread 17 is running
Thread 16 is running

Thread Class vs Runnable Interface

1. If we extend the Thread class, our class cannot extend any other class because Java doesn't support multiple inheritance. But, if we implement the Runnable interface, our class can still extend other base classes.

2. We can achieve basic functionality of a thread by extending Thread class because it provides some inbuilt methods like yield(), interrupt() etc. that are not available in Runnable interface.

3. Using runnable will give you an object that can be shared amongst multiple threads.

Table 12.1: Java Thread Class Methods

start()	The start method initiates the execution of a thread
currentThread()	The currentThread method returns the reference to the currently executing thread object.
run()	The run method triggers an action for the thread
isAlive()	The isAlive method is invoked to verify if the thread is alive or dead
sleep()	The sleep method is used to suspend the thread temporarily
yield()	The yield method is used to send the currently executing threads to standby mode and runs different sets of threads on higher priority
suspend()	The suspend method is used to instantly suspend the thread execution
resume()	The resume method is used to resume the execution of a suspended thread only
interrupt()	The interrupt method triggers an interruption to the currently executing thread class
destroy()	The destroy method is invoked to destroy the execution of a group of threads
stop()	The stop method is used to stop the execution of a thread

Priorities in threads: Priorities in threads is a concept where each thread is having a priority which in layman's language one can say every object is having priority here which is represented by numbers ranging from 1 to 10.

The default priority is set to 5 as excepted.

Minimum priority is set to 1.

Maximum priority is set to 10.

Here 3 constants are defined in it namely as follows:

```
public static int NORM_PRIORITY
```

```
public static int MIN_PRIORITY
```

```
public static int MAX_PRIORITY
```

Get and Set priority of a thread in java.

`public final int getPriority(): java.lang.Thread.getPriority()` method returns priority of given thread.

`public final void setPriority(int newPriority): java.lang.Thread.setPriority()` method changes the priority of thread to the value `newPriority`. This method throws `IllegalArgumentException` if value of parameter `newPriority` goes beyond minimum(1) and maximum(10) limit.

Example:

```
import java.lang.*;
class ThreadDemo extends Thread
{
    public void run()
    {
        System.out.println("Inside run method");
    }
}
public static void main(String[] args)
{
    ThreadDemo t1 = new ThreadDemo();
    ThreadDemo t2 = new ThreadDemo();
    System.out.println("t1 thread priority : " + t1.getPriority());
    System.out.println("t2 thread priority : " + t2.getPriority());
    t1.setPriority(2);
    t2.setPriority(5);
    System.out.println("t1 thread priority : " + t1.getPriority());
    System.out.println("t2 thread priority : " + t2.getPriority());
    System.out.println("Currently Executing Thread : " +
        Thread.currentThread().getName());
    System.out.println("Main thread priority : " +
        Thread.currentThread().getPriority());
    Thread.currentThread().setPriority(10);
    System.out.println("Main thread priority : " +
        Thread.currentThread().getPriority());
}
t1 thread priority : 5
t2 thread priority : 5
t1 thread priority : 2
t2 thread priority : 5
Currently Executing Thread : main
Main thread priority : 5
Main thread priority : 10
```

10.6 Procedure:

Import the necessary packages for multithreading in java, design two or more classes which will work as thread, in each class write a run() method which will convey the objective of thread. From main class activate the threads with the help of start keyword.

10.7 Program and Output:

```
// Thread 1: Prints numbers 1 to 5
class NumberThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Number: " + i);
            try {
                Thread.sleep(500); // pause for clarity
            } catch (InterruptedException e) {
                System.out.println("NumberThread interrupted");
            }
        }
    }
}

// Thread 2: Prints alphabets A to E
class AlphabetThread extends Thread {
    public void run() {
        for (char c = 'A'; c <= 'E'; c++) {
            System.out.println("Alphabet: " + c);
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("AlphabetThread interrupted");
            }
        }
    }
}

// Thread 3: Prints symbol * five times
class SymbolThread extends Thread {
    public void run() {
        for (int i = 0; i < 5; i++) {
            System.out.println("Symbol: *");
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println("SymbolThread interrupted");
            }
        }
    }
}
```

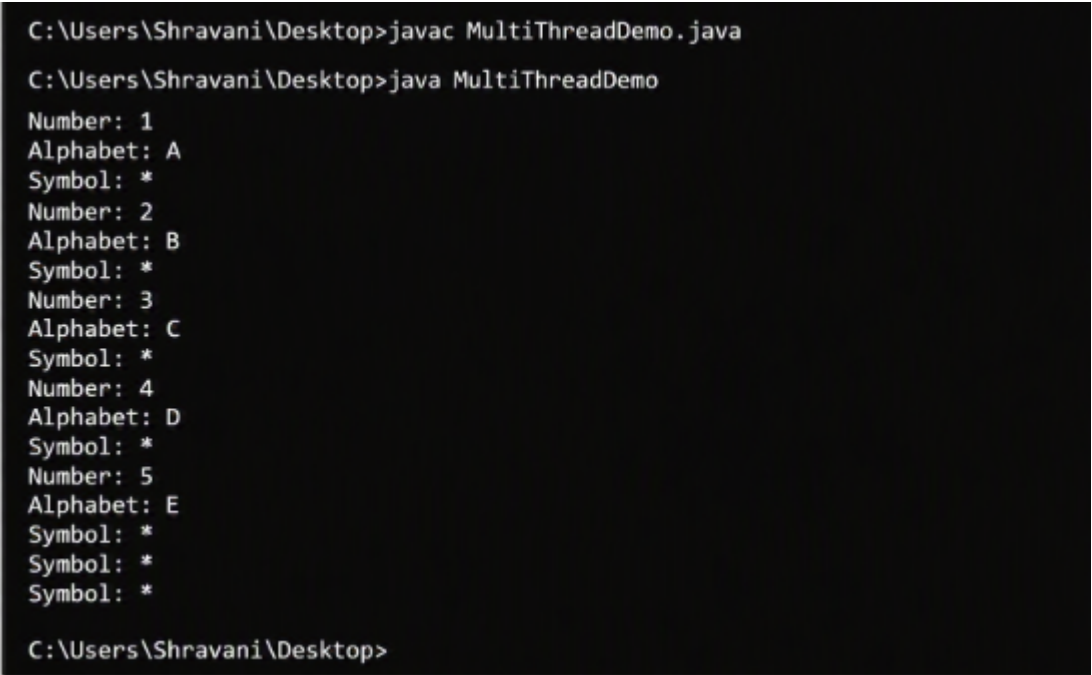
```

    }
}

// Main class
public class MultiThreadDemo {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread();
        AlphabetThread t2 = new AlphabetThread();
        SymbolThread t3 = new SymbolThread();

        t1.start();
        t2.start();
        t3.start();
    }
}

```



```

C:\Users\Shravani\Desktop>javac MultiThreadDemo.java
C:\Users\Shravani\Desktop>java MultiThreadDemo
Number: 1
Alphabet: A
Symbol: *
Number: 2
Alphabet: B
Symbol: *
Number: 3
Alphabet: C
Symbol: *
Number: 4
Alphabet: D
Symbol: *
Number: 5
Alphabet: E
Symbol: *
Symbol: *
Symbol: *
C:\Users\Shravani\Desktop>

```

10.8 Conclusion:

This experiment demonstrates how multithreading allows multiple tasks to run concurrently, improving CPU utilization and efficiency. By creating three threads (numbers, alphabets, and symbols), we see how independent tasks can execute simultaneously without blocking each other. This highlights the importance of thread management in building responsive and efficient applications.

10.9 Questions:

1. What is a thread?

A thread is the smallest unit of execution in a program. It is a lightweight sub-process that runs independently but shares memory with other threads in the same process.

2. Differentiate between process and thread.

Aspect	Process	Thread
Definition	Independent program in execution	Smallest unit of execution within a process
Memory	Has its own memory space	Shares memory with other threads of the same process
Overhead	Heavy, requires more resources	Lightweight, less resource-intensive
Communication	Inter-process communication is complex	Threads communicate easily via shared memory

3. What is the purpose of `wait()` method in Java?

- The `wait()` method is used to pause a thread until another thread notifies it using `notify()` or `notifyAll()`.
- It helps in synchronization when multiple threads are working on shared resources.

4. What are the states in the lifecycle of a Thread?

New: Thread is created but not started.

Runnable: Thread is ready to run but waiting for CPU scheduling.

Running: Thread is actively executing.

Waiting/Blocked: Thread is paused, waiting for another thread or resource.

Dead/Terminated: Thread has finished execution.